



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251233
Nama Lengkap	Mikael Gratianus Satrio Adi Kuncoro
Minggu ke / Materi	12 / Tipe Data Dictionary

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

Dictionary mirip dengan list, tetapi lebih fleksibel karena menggunakan key sebagai pengganti indeks integer. Dictionary berisi pasangan **key:value**, di mana setiap key bersifat unik dan digunakan untuk mencari value. Contohnya seperti kamus bahasa, kata menjadi key dan artinya menjadi value.

```
>>> x = dict()
>>> print(x)
{}
>>> x ['one'] = 'uno'
>>> print(x)
{'one': 'uno'}
>>>
```

Gambar 1.1: Penerapan dict dalam python

Perhatikan Gambar 1.1. Fungsi **dict** digunakan untuk membuat dictionary baru yang kosong. Karena **built-in function** dari python maka penggunaannya perlu dihindari sebagai nama variabel. Tanda kurung kurawal {} digunakan untuk merepresentasikan dictionary kosong. Jika ingin menambahkan item dalam dictionary, dapat menggunakan kurung kotak []. Kode diatas (Gambar 1.1) membuat item yang memetakan key **'one'** ke value **'uno'**. Saat dictionary dicetak, akan terlihat pasangan key dan value tersebut.

Format output pada dictionary juga bisa digunakan sebagai format input. Misalnya, saat membuat dictionary baru lalu mencetaknya, hasil yang tampil adalah seluruh isi dictionary tersebut. Lihat Gambar 1.2.

```
>>> x = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(x)
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(x['two'])
dos
```

Gambar 1.2: Penerapan dict dalam pemanggilan value dengan print key

Urutan item pada dictionary tidak selalu tetap atau dapat diprediksi. Namun ini bukan masalah karena dictionary tidak menggunakan indeks integer, melainkan menggunakan key untuk mencari value yang sesuai. Kunci **'two'** selalu dipasangkan dengan nilai **'dos'** jadi urutan pada item tidak terlalu berpengaruh. Jika menggunakan kunci yang tidak ada dalam dictionary, akan didapatkan pengecualian, berikut contohnya pada Gambar 1.3.

```

>>> x = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(x)
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(x['four'])
Traceback (most recent call last):
  File "<python-input-8>", line 1, in <module>
    print(x['four'])
          ~^^^^^^^^
KeyError: 'four'

```

Gambar 1.3: Contoh output jika menggunakan **key** yang tidak sesuai

```

>>> x = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(x)
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> len(x)
3
>>> 'one' in x
True
>>> 'uno' in x
False
>>> values = list(x.values())
>>> 'uno' in values
True

```

Gambar 1.4: Contoh penggunaan fungsi **len**, operator **in**, dan metode **values**

Liat Gambar 1.4, fungsi **len** pada dictionary digunakan untuk menghitung jumlah pasangan key-value. Operator **in** digunakan untuk mengecek apakah suatu key ada dalam dictionary. Sedangkan method **values** digunakan untuk mengambil seluruh value dalam dictionary. Operator **in** pada *list* menggunakan pencarian linear sehingga semakin banyak isi *list*, semakin lama proses pencariannya. Sedangkan pada *dictionary*, python menggunakan algoritma *Hash Table* sehingga pencarian tetap cepat meskipun jumlah item bertambah banyak.

Dictionary sebagai set penghitung (counters)

Sebagai contoh kita diberikan sebuah string dan di haruskan menghitung banyaknya huruf yang muncul, beberapa cara yang bisa digunakan misalnya:

1. Membuat 26 variabel untuk setiap huruf alfabet dan menghitung kemunculannya menggunakan percabangan.
2. Membuat list berisi 26 elemen, lalu mengubah karakter menjadi indeks angka untuk menghitung kemunculannya.
3. Membuat dictionary dengan karakter sebagai key dan jumlah kemunculannya sebagai value.

Ketiga opsi tersebut melakukan perhitungan yang sama, tetapi dengan cara yang berbeda. Dibandingkan metode lain, penggunaan dictionary lebih praktis karena tidak perlu mengetahui huruf mana yang akan muncul dalam string. Kita hanya perlu memberikan ruang untuk huruf yang akan muncul. Model penerapannya sebagai berikut:

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] = d[c] + 1
8 print(d)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\dicti
onary_lat.py"
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
PS D:\New folder - Copy\Coding>
```

Gambar 1.5: Penerapan dictionary histogram

Model komputasi diatas disebut dengan histogram atau istilah statistika nya adalah set perhitungan. Histogram output menunjukkan bahwa huruf "a" dan "b" muncul sekali; "o" muncul dua kali, dan seterusnya. Perulangan for memeriksa setiap karakter pada string. Jika karakter belum ada di dictionary, nilainya dibuat 1. Jika sudah ada, nilainya ditambah 1. Output histogram menunjukkan jumlah kemunculan tiap huruf.

Metode `get` pada dictionary digunakan untuk mengambil nilai dari sebuah kunci. Jika kunci ada, nilainya dikembalikan. Jika tidak, akan mengembalikan nilai default.

```
>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0
```

Gambar 1.6: Penerapan `get` pada dictionary

Metode **`get`** membuat loop histogram lebih ringkas karena otomatis menangani kunci yang belum ada, sehingga **`if`** tidak diperlukan.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     d[c] = d.get(c,0) + 1
5 print(d)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\dicti
onary_lat.py"
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
PS D:\New folder - Copy\Coding>
```

Gambar 1.7: Penerapan `get` dictionary histogram

Penggunaan **`get`** untuk penghitungan menjadi idiom umum di python karena lebih ringkas dibanding penggunaan **`if`** dan operator **`in`**, meskipun hasilnya sama.

Dictionary dan File

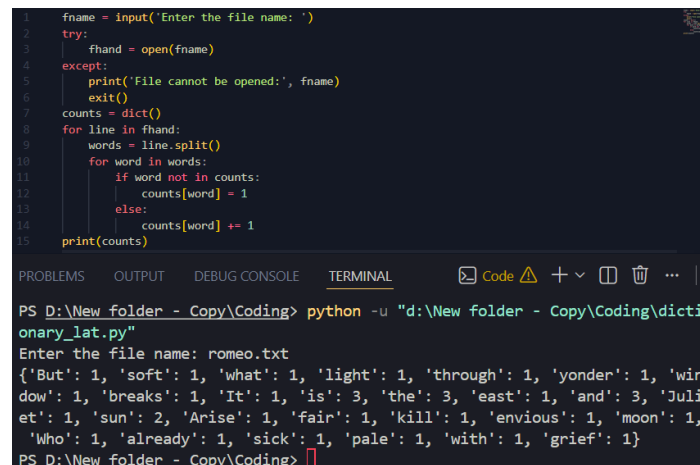
Salah satu kegunaan dictionary adalah untuk menghitung kemunculan kata-kata dalam file dengan beberapa teks tertulis. Sebagai contoh pertama, akan digunakan versi

teks yang disingkat dan disederhanakan tanpa tanda baca. Selanjutnya akan digunakan teks adegan dengan tanda baca yang disertakan.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Gambar 1.8: Contoh romeo.txt

Untuk kasus ini program python menggunakan nested loop (perulangan bersarang) untuk menghitung jumlah kata dalam file. Perulangan luar membaca setiap baris file, sedangkan perulangan dalam memeriksa setiap kata pada baris tersebut. Setiap kata kemudian dihitung dan disimpan ke dalam dictionary, sehingga seluruh kata dalam file dapat diproses dan dihitung jumlah kemunculannya.



```
1 fname = input('Enter the file name: ')
2 try:
3     fhand = open(fname)
4 except:
5     print('File cannot be opened:', fname)
6     exit()
7 counts = dict()
8 for line in fhand:
9     words = line.split()
10    for word in words:
11        if word not in counts:
12            counts[word] = 1
13        else:
14            counts[word] += 1
15 print(counts)
```

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\dictionary_lat.py"

Enter the file name: romeo.txt

```
{'But': 1, 'soft': 1, 'what': 1, 'light': 1, 'through': 1, 'yonder': 1, 'win
dow': 1, 'breaks': 1, 'It': 1, 'is': 3, 'the': 3, 'east': 1, 'and': 3, 'Juli
et': 1, 'sun': 2, 'Arise': 1, 'fair': 1, 'kill': 1, 'envious': 1, 'moon': 1,
'Who': 1, 'already': 1, 'sick': 1, 'pale': 1, 'with': 1, 'grief': 1}
```

PS D:\New folder - Copy\Coding>

Gambar 1.9: Penerapan dictionary dan file dengan source code python

Pada statement else digunakan model penulisan yang lebih singkat untuk menambahkan variable. **Counts[word] += 1** adalah penulisan singkat dari **counts[word] = counts[word] + 1**. Cara serupa juga dapat digunakan dengan operator lain seperti -=, *=, dan /= untuk mengubah nilai variabel.

Looping dan Dictionary

Pada perulangan for, dictionary ditelusuri berdasarkan kuncinya, lalu setiap kunci dan nilainya dapat dicetak sesuai pasangannya.

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     print(key, counts[key])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL Code ⚠ + v

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\onary_lat.py"

chuck 1
annie 42
jan 100
PS D:\New folder - Copy\Coding>

Gambar 2.0: Kode sederhana fungsi for untuk menelusuri kunci yang ada didalam nya

Jika ingin menemukan semua entri dalam dictionary dengan nilai diatas 10, maka kode programnya sebagai berikut:

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     if counts[key] > 10 :
4         print(key, counts[key])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL Code ⚠ + v | | | |

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\dicti onary_lat.py"

annie 42
jan 100
PS D:\New folder - Copy\Coding>

Gambar 2.1: Source code menemukan semua entri dalam dictionary dengan nilai >10

Pada looping for yang melalui kunci dictionary, perlu ditambahkan opertor indeks untuk mengambil nilai yang sesuai untuk setiap kunci.

Program diatas hanya akan menampilkan data nilai diatas 10. Untuk mencetak kunci dictionary secara alfabet, cara nya yaitu dengan kunci diubah menjadi list, kemudian diurutkan dengan **sort()**, lalu dilakukan perulangan untuk menampilkan setiap kunci beserta nilainya. Impelentasi dalam kode nya sebagai berikut:

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 lst = list(counts.keys())
3 print(lst)
4 lst.sort()
5 for key in lst:
6     print(key, counts[key])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL Code ⚠

PS D:\New folder - Copy\Coding> python -u "d:\New folder - C onary_lat.py"

['chuck', 'annie', 'jan']
annie 42
chuck 1
jan 100
PS D:\New folder - Copy\Coding>

Gambar 2.2: Penerapan print key dictionary secara alfabet

Advenced Text Parsing

Pada bagian ini digunakan file romeo.txt yang masih memiliki tanda baca lengkap untuk melanjutkan contoh pengolahan kata pada file.

```

But, soft! what light through yonder window breaks?
It is the east, and Juliet is the sun.
Arise, fair sun, and kill the envious moon,
Who is already sick and pale with grief,

```

Gambar 2.3: Contoh romeo.txt (2)

Pada python fungsi `split()` untuk membagi teks menjadi kata-kata berdasarkan spasi. Akibatnya, kata seperti "**soft**" dan "**soft!**" atau "**who**" dan "**Who**" akan dianggap berbeda dan dihitung terpisah dalam dictionary. Untuk menyamakan penulisan kata, dapat digunakan method seperti **lower**, **punctuation** dan **translate**. Jika diperhatikan method string `translate` merupakan method string yang paling subtle (hampir tidak kentara). Berikut ini merupakan dokumentasi dari method `translate`.

```
>>> line.translate(str.maketrans(fromstr, tostr, deletetr))
```

Ganti karakter pada **fromstr** dengan karakter pada posisi yang sama dengan **tostr** dan hapus semua karakter yang ada dalam **deletetr**. Untuk **fromstr** dan **tostr** dapat berupa string kosong dan untuk parameter pada **deletetr** dapat dihilangkan.

Ini tidak menggunakan **tostr**, tetapi memakai **deletetr** untuk menghapus semua tanda baca yang secara otomatis dikenali oleh Python.

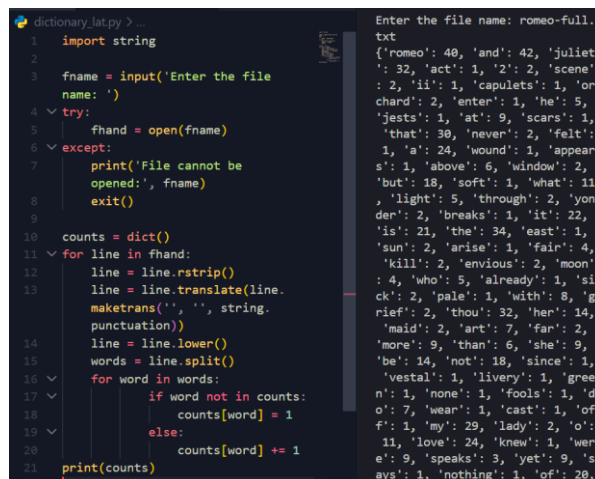
```

>>> import string
>>> string.punctuation
'!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~'
>>>

```

Gambar 2.4: Contoh string punctuation

Penggunaan dalam program python sebagai berikut:



```

dictionary_lat.py > ...
1 import string
2
3 fname = input('Enter the file
name: ')
4 try:
5     fhand = open(fname)
6 except:
7     print('File cannot be
opened:', fname)
8     exit()
9
10 counts = dict()
11 for line in fhand:
12     line = line.rstrip()
13     line = line.translate(line.
maketrans(' ', ' ', string.
punctuation))
14     line = line.lower()
15     words = line.split()
16     for word in words:
17         if word not in counts:
18             counts[word] = 1
19         else:
20             counts[word] += 1
21 print(counts)

```

```

Enter the file name: romeo-full.
txt
{'romeo': 40, 'and': 42, 'juliet': 32, 'act': 1, '2': 2, 'scene': 2, 'ii': 1, 'capulets': 1, 'on': 1, 'chard': 2, 'enter': 1, 'he': 5, 'jests': 1, 'at': 9, 'scars': 1, 'that': 30, 'never': 2, 'felt': 1, 'a': 24, 'wound': 1, 'appear': 1, 'above': 6, 'window': 2, 'but': 18, 'soft': 1, 'what': 11, 'light': 5, 'through': 2, 'yonder': 2, 'breaks': 1, 'it': 22, 'is': 21, 'the': 34, 'east': 1, 'sun': 2, 'arise': 1, 'fair': 4, 'kill': 2, 'envious': 2, 'moon': 4, 'who': 5, 'already': 1, 'sick': 2, 'pale': 1, 'with': 8, 'grief': 2, 'thou': 32, 'her': 14, 'maid': 2, 'art': 7, 'fan': 2, 'more': 9, 'than': 6, 'she': 9, 'be': 14, 'not': 18, 'since': 1, 'vestal': 1, 'livery': 1, 'green': 1, 'none': 1, 'fools': 1, 'do': 7, 'wear': 1, 'cast': 1, 'of': 1, 'my': 29, 'lady': 2, 'o': 11, 'love': 24, 'knew': 1, 'were': 9, 'speaks': 3, 'yet': 9, 'says': 1, 'nothing': 1, 'of': 20,

```

Gambar 2.5: Contoh penerapan nya dalam python

LINK GITHUB

: <https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git>

SOAL 01



```
soal01.py > ...
1 dictionary = {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}
2 print(f"{'key':<8} {'value':<8} {'item':<8}")
3 for kunci, nilai in dictionary.items():
4     print(f"{'kunci':<8} {'nilai':<8} {'kunci':<8}")
5

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Code + v [] [] ... |

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu12\soal01.py"
key      value    item
1        10      1
2        20      2
3        30      3
4        40      4
5        50      5
6        60      6
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12>
```

Gambar 2.6: Source code soal 01

Penjelasan:

Ini adalah program untuk mendapatkan nilai key, value, dan item dari sebuah dictionary. Pertama membuat sebuah dictionary dengan variabel **dictionary** yang berisi 6 pasangan key-value, yaitu key 1 hingga 6 dengan value 10 hingga 60. Setelah itu, kode mencetak header tabel menggunakan format rata kiri dengan lebar kolom 8 karakter untuk kolom key, value, dan item. Selanjutnya dilakukan perulangan menggunakan **dictionary.items()** untuk mengambil key dan value secara bersamaan, di mana variabel **kunci** menyimpan key dan variabel **nilai** menyimpan value. Pada setiap perulangan, program mencetak isi tabel dengan format yang sama, yaitu kolom key berisi nilai dari kunci, kolom value berisi nilai dari nilai, dan kolom item juga menampilkan nilai dari kunci.

SOAL 02



```
soal02.py > ...
1  lista = ['red', 'green', 'blue']
2  listb = ['#FF0000', '#008000', '#0000FF']
3  x = dict(zip(lista, listb))
4  sorting_kunci= sorted(x.keys(), key=len, reverse=True)
5  hasil = {k: x[k] for k in sorting_kunci}
6  print(hasil)
~

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu12\soal02.py"
{"green": "#008000", "blue": "#0000FF", "red": "#FF0000"}
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12>
```

Gambar 2.7: Source code soal 02

Penjelasan:

Ini adalah program untuk memetakan dua list menjadi satu dictionary. Pertama Kode tersebut membuat dua buah list, yaitu **lista** yang berisi nama warna ('red', 'green', 'blue') dan **listb** yang berisi kode warna yang sesuai dengan masing-masing warna. Selanjutnya, fungsi **zip(lista, listb)** digunakan untuk menggabungkan kedua list secara berpasangan, kemudian hasilnya diubah menjadi dictionary menggunakan **dict()**, sehingga terbentuk pasangan seperti 'red': '#FF0000', 'green': '#008000', dan 'blue': '#0000FF'. Setelah itu, program mengurutkan key pada dictionary berdasarkan panjang karakter menggunakan **sorted(x.keys(), key=len, reverse=True)** dari yang terpanjang ke terpendek, sehingga menghasilkan urutan ['green', 'blue', 'red']. Urutan tersebut kemudian digunakan dalam dictionary comprehension untuk membuat dictionary baru bernama hasil dengan susunan key mengikuti hasil pengurutan, sehingga menjadi {'green': '#008000', 'blue': '#0000FF', 'red': '#FF0000'}. Terakhir, program mencetak isi dictionary hasil ke terminal.

SOAL 03



```
soal03.py > ...
1  nama_file = input("Masukkan nama file : ")
2  try:
3      handle = open(nama_file)
4      x = dict()
5      for baris in handle:
6          if baris.startswith('From '):
7              kata = baris.split()
8              email = kata[1]
9              x[email] = x.get(email, 0) + 1
10         print(x)
11 except FileNotFoundError:
12     print(f"File {nama_file} tidak ditemukan.")

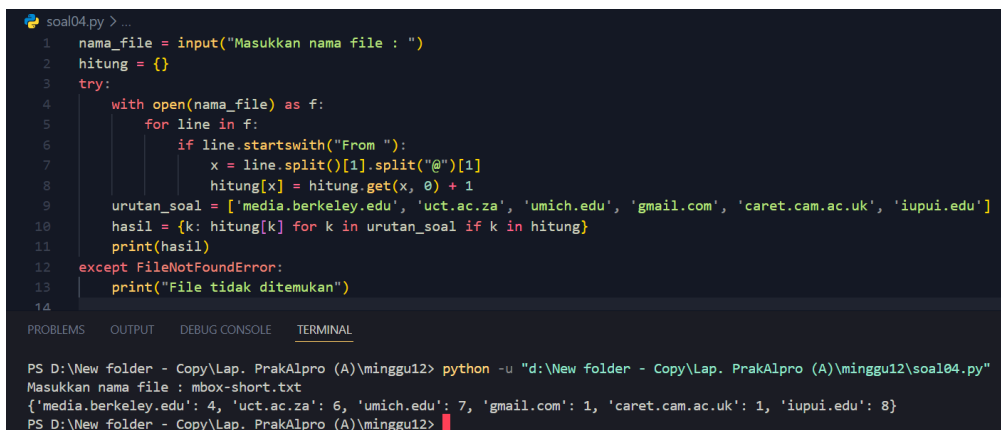
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu12\soal03.py"
Masukkan nama file : mbox-short.txt\
{'gopal.ramasammycook@gmail.com': 1, 'louis@media.berkeley.edu': 3, 'cwen@iupui.edu': 5, 'antranig@caret.cam.ac.uk': 1, 'rjlowe@iupui.edu': 2, 'gsilver@umich.edu': 3, 'david.horwitz@uct.ac.za': 4, 'wagnerm@iupui.edu': 1, 'zqian@umich.edu': 4, 'stephen.marquard@uct.ac.za': 2, 'ray@media.berkeley.edu': 1}
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12>
```

Gambar 2.8: Source code soal 03

Penjelasan:

Ini adalah program dapat membaca log email dan sajikan dalam histogram menggunakan dictionary dan menghitung berapa banyak pesan yang masuk dari email dan sajikan dalam bentuk dictionary. Pertama kode dibuat untuk meminta pengguna memasukkan nama file yang akan dibaca lalu menyimpannya ke variabel **nama_file**. Program kemudian menggunakan blok **try-except** untuk menangani kemungkinan error saat membuka file. Pada bagian try, file dibuka menggunakan **open()** dan disimpan ke variabel **handle**, kemudian dibuat dictionary kosong **x** untuk menghitung jumlah kemunculan email. Program membaca file baris demi baris menggunakan perulangan **for**, lalu memeriksa apakah suatu baris diawali dengan teks **'From '**. Jika sesuai, baris tersebut dipecah menjadi beberapa kata menggunakan **split()**, kemudian alamat email pengirim diambil dari elemen index ke-1 dan disimpan ke variabel **email**. Selanjutnya, dictionary **x** digunakan untuk menghitung berapa kali setiap email muncul dengan bantuan **x.get(email, 0) + 1**, sehingga jika email belum ada nilainya dianggap 0 lalu ditambah 1. Setelah seluruh file selesai diproses, program mencetak dictionary yang berisi daftar email beserta jumlah kemunculannya. Jika file yang dimasukkan tidak ditemukan, blok except **FileNotFoundError** akan dijalankan untuk menampilkan pesan bahwa file tersebut tidak ditemukan.

SOAL 04



```
soal04.py > ...
1  nama_file = input("Masukkan nama file : ")
2  hitung = {}
3  try:
4      with open(nama_file) as f:
5          for line in f:
6              if line.startswith("From "):
7                  x = line.split()[1].split("@")[1]
8                  hitung[x] = hitung.get(x, 0) + 1
9          urutan_soal = ['media.berkeley.edu', 'uct.ac.za', 'umich.edu', 'gmail.com', 'caret.cam.ac.uk', 'iupui.edu']
10         hasil = {k: hitung[k] for k in urutan_soal if k in hitung}
11         print(hasil)
12     except FileNotFoundError:
13         print("File tidak ditemukan")
14

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu12\soal04.py"
Masukkan nama file : mbox-short.txt
{'media.berkeley.edu': 4, 'uct.ac.za': 6, 'umich.edu': 7, 'gmail.com': 1, 'caret.cam.ac.uk': 1, 'iupui.edu': 8}
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu12>
```

Gambar 2.9: Source code soal 04

Penjelasan:

Ini adalah program untuk mencatat data nama domain pengirim pesan dan menghitung jumlah pesan yang dikirim masing-masing domain yang disajikan dalam bentuk dictionary. Pertama kode dibuat untuk meminta pengguna memasukkan nama file yang akan dibaca lalu menyimpannya ke variabel **nama_file**. Setelah itu, program membuat dictionary kosong bernama **hitung** untuk menghitung jumlah kemunculan domain email dan menggunakan blok **try-except** agar program tidak berhenti jika terjadi error. Pada bagian try, file dibuka menggunakan **with open(nama_file) as f**, sehingga file akan otomatis ditutup setelah selesai digunakan. Program kemudian membaca file satu baris demi satu baris menggunakan perulangan **for**, lalu hanya memproses baris yang diawali dengan teks "**From**". Dari baris tersebut, program mengambil alamat email menggunakan **split()[1]**, kemudian mengambil bagian domain setelah tanda **@** dengan **split("@")[1]**. Domain yang diperoleh disimpan ke variabel **x**, lalu jumlah kemunculannya dihitung menggunakan **hitung[x] = hitung.get(x, 0) + 1**, sehingga jika domain belum ada di dictionary nilainya dianggap 0 lalu ditambah 1. Selanjutnya dibuat list **urutan_soal** yang berisi domain-domain tertentu sekaligus menentukan urutan tampilannya. Program kemudian membuat dictionary baru bernama **hasil** menggunakan dictionary comprehension, yang hanya berisi domain-domain dari **urutan_soal** yang benar-benar ada di dictionary **hitung**, dengan urutan sesuai list tersebut. Setelah itu, program mencetak dictionary **hasil** ke terminal. Jika file yang dimasukkan tidak ditemukan, maka blok except **FileNotFoundError** akan dijalankan untuk menampilkan pesan "File tidak ditemukan".